



2D Array

In Java, 2D array is an array of arrays

A 2D array in Java, also known as a **multidimensional** array, is essentially an array of arrays.

It provides a way to store data in a **tabular format**, resembling rows and columns like a matrix or a spreadsheet

Declaration

To declare a 2D array,
Specify the data type followed by two sets of
square brackets, then the array name.

```
int[][] matrix; // Declares an array of integers
```

```
String[][] names; // Declares an array of strings
```

Initialization

To create a fixed-size 2D array,

- Use the new keyword,
- Specify the number of rows and columns.

```
int[][] matrix = new int[3][4]; //Creates a 3x4 integer  
matrix
```

Initialization (Literal Values)

initialize a 2D array with values directly during declaration.
(similarly as 1D arrays)

```
int[][] x = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};
```

	Column 0	Column 1	Column 2
Row 0	x[0][0]	x[0][1]	x[0][2]
Row 1	x[1][0]	x[1][1]	x[1][2]
Row 2	x[2][0]	x[2][1]	x[2][2]

Jagged Arrays

In Java, each row of a 2D array can have a different length, creating a "jagged array."

```
int[][] jaggedArray = new int[3][]; // Declare with fixed  
rows, but not columns  
jaggedArray[0] = new int[2]; // Row 0 has 2 columns  
jaggedArray[1] = new int[4]; // Row 1 has 4 columns  
jaggedArray[2] = new int[1]; // Row 2 has 1 column
```

Putting data into a 2D array

```
int[][] arr = new int[3][4];  
arr[0][1] = 10;
```

Putting data into a 2D array

```
int[][] arr = new int[3][4];  
arr[0][1] = 10;
```

0	10	0	0
0	0	0	0
0	0	0	0

We use square brackets to access a specific spot in the array.

This is called *indexing into* the array.

Arrays hold different values at each location.

Putting data into a 2D array

```
// Declare and create a 2D array with 3 rows and 4 columns  
String[][] names = new String[3][4];
```

```
// Assign values to specific elements
```

```
names[0][0] = "Alice";  
names[0][1] = "Bob";  
names[1][2] = "Charlie";  
names[2][3] = "David";
```

"Alice"	"Bob"	null	null
null	null	"Charlie"	null
null	null	null	"David"

Getting data out of a 2D array

```
int[][] arr = new int[3][4];  
arr[0][1] = 10;
```

```
System.out.println(arr[0][1]); //this prints 10
```

The length of a 2D array

To determine **length**, consider both the number of rows and the number of columns.

To get the number of rows in a 2D array, you use the `.length` property on the array itself. This returns the length of the outer array, which represents the number of rows.

```
int[][] myArray = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};  
int numRows = myArray.length; // numRows will be 3
```

The length of a 2D array

To get the number of columns in a specific row, you access that particular row (which is an array itself) and then apply the `.length` property to it. Note that in Java, rows in a 2D array can have different lengths (jagged arrays).

```
int[][] myArray = {{1, 2, 3}, {4, 5}, {6, 7, 8, 9}};  
int numColIn1stRow = myArray[0].length; //numColIn1stRow will be 3  
int numColIn2ndRow = myArray[1].length; //numColIn2ndRow will be 2
```

If you are working with a rectangular 2D array where all rows have the same number of columns, you can simply check the length of the first row (or any row) to get the column count for the entire array

What is `int[][]`, primitive or reference?

A: primitive type

B: reference type

What is `int[][]`, primitive or reference?

A: primitive type

B: reference type

All arrays are reference types 1D or 2D, even if they hold primitive type data elements.

```
int[][] nums = {  
    {5, 6, 7},  
    {8, 9, 10}  
};
```

What is `nums[1][0]`?

- A. 5
- B. 6
- C. 8
- D. 9
- E. 10

```
int[][] nums = {  
    {5, 6, 7},  
    {8, 9, 10}  
};
```

What is `nums[1][0]`?

A. 5

B. 6

C. 8

D. 9

E. 10

What does `nums.length` represent?

- A. Total number of elements
- B. Number of rows
- C. Number of columns
- D. Always equal to 3
- E. None of the above

What does `nums.length` represent?

- A. Total number of elements
- B. Number of rows
- C. Number of columns
- D. Always equal to 3
- E. None of the above

```
int[][] nums = {  
    {5, 6, 7},  
    {8, 9, 10}  
};
```

What does `nums[0].length` return?

- A. 1
- B. 2
- C. 3
- D. 4
- E. 6

```
int[][] nums = {  
    {5, 6, 7},  
    {8, 9, 10}  
};
```

What does `nums[0].length` return?

- A. 1
- B. 2
- C. 3
- D. 4
- E. 6

```
int[][] nums = {  
    {5, 6, 7},  
    {8, 9, 10}  
};  
System.out.println(nums[2][0]);
```

What does it print?

- A. Prints 0
- B. Prints null
- C. Throws an exception
- D. Prints 5
- E. Compile error

```
int[][] nums = {  
    {5, 6, 7},  
    {8, 9, 10}  
};  
System.out.println(nums[2][0]);
```

What does it print?

- A. Prints 0
- B. Prints null
- C. Throws an exception "ArrayIndexOutOfBoundsException"
- D. Prints 5
- E. Compile error

```
int[][] nums = {  
    {5, 6, 7},  
    {8, 9, 10}  
};  
nums[0][2] = nums[1][1];
```

What does this code do?

- A. Replaces 7 with 9
- B. Replaces 9 with 7
- C. Adds two numbers
- D. Does nothing
- E. Replaces 7 with 8

```
int[][] nums = {  
    {5, 6, 7},  
    {8, 9, 10}  
};  
nums[0][2] = nums[1][1];
```

What does this code do?

- A. Replaces 7 with 9
- B. Replaces 9 with 7
- C. Adds two numbers
- D. Does nothing
- E. Replaces 7 with 8


```
int[][] data = {  
    {1, 2},  
    {3, 4, 5}  
};  
System.out.println(data[1].length);
```

What is printed?

- A. 2
- B. 3
- C. 4
- D. 5
- E. 6

```
int[][] data = {  
    {1, 2},  
    {3, 4, 5}  
};  
System.out.println(data[1].length);
```

What is printed?

- A. 2
- B. 3
- C. 4
- D. 5
- E. 6

```
int[][] grid = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};  
for (int i = 0; i < grid.length; i++)  
    System.out.print(grid[i][i] + " ");
```

What is printed?

- A. 1 5 9
- B. 3 5 7
- C. 2 5 8
- D. 9 5 1
- E. 1 4 7

```
int[][] grid = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};  
for (int i = 0; i < grid.length; i++)  
    System.out.print(grid[i][i] + " ");
```

What is printed?

A. 1 5 9

B. 3 5 7

C. 2 5 8

D. 9 5 1

E. 1 4 7

```
int[][] a = {  
    {2, 4},  
    {6, 8}  
};  
int sum = 0;  
for (int i = 0; i < a.length; i++)  
    sum += a[i][1];  
System.out.println(sum);
```

What is printed?

- A. 6
- B. 8
- C. 12
- D. 14
- E. 10

```
int[][] a = {  
    {2, 4},  
    {6, 8}  
};  
int sum = 0;  
for (int i = 0; i < a.length; i++)  
    sum += a[i][1];  
System.out.println(sum);
```

What is printed?

- A. 6
- B. 8
- C. 12
- D. 14
- E. 10

```
for (int row = 0; row < grid.length; row++) {  
    for (int col = 0; col < grid[row].length; col++)  
    {  
        System.out.print(grid[row][col] + " ");  
    }  
    System.out.println();  
}
```

How many times will the inner loop run if grid has 4 rows and 3 columns?

- A. 4
- B. 3
- C. 7
- D. 12
- E. 16

```
for (int row = 0; row < grid.length; row++) {  
    for (int col = 0; col < grid[row].length; col++)  
    {  
        System.out.print(grid[row][col] + " ");  
    }  
    System.out.println();  
}
```

How many times will the inner loop run if grid has 4 rows and 3 columns?

- A. 4
- B. 3
- C. 7
- D. 12 ($4 \times 3 = 12$)**
- E. 16


```
int sum = 0;
for (int i = 0; i < grid.length; i++) {
    for (int j = 0; j < grid[i].length; j++) {
        sum += grid[i][j];
    }
}
```

What will **sum** contain if `grid = {{1,2},{3,4}}`?

- A. 12
- B. 10
- C. 8
- D. 6
- E. 4

```
int sum = 0;
for (int i = 0; i < grid.length; i++) {
    for (int j = 0; j < grid[i].length; j++) {
        sum += grid[i][j];
    }
}
```

What will **sum** contain if `grid = {{1,2},{3,4}}`?

A. 12

B. 10 (1 + 2 + 3 + 4 = 10)

C. 8

D. 6

E. 4

Common Mistakes

- Mixing up row and column order
~~(array[col][row])~~ (array[row][col])
- Forgetting zero-based indexing
- Uneven row lengths (in “jagged arrays”)

Real-World Example

2D arrays can represent:

- Images (pixels)
- Game boards (e.g., Tic-Tac-Toe)
- Tables of data

Summary

- 2D array = array of arrays
- Access with [row][col]
- Use nested loops for traversal
- Watch out for jagged arrays and indexing errors